

Lab 3 Report: Image Manipulations using OpenCV

Student name : Mujtaba Hejji Alnuwaysir

ID : 2240007941

Class : 7MA1

Course: ARTI 404 – Image Processing

Session: Lab 3 – Image Manipulations using OpenCV Library

1. Introduction

This lab focuses on understanding how digital images are represented and manipulated using the OpenCV library in Python. The main objectives were to explain how images are stored as numerical arrays, and to apply geometric and intensity transformations to a sample image.

2. Tools & Libraries

- **Language:** Python 3
- **IDE:** Visual Studio Code (Jupyter Notebook extension)
- **Libraries:** OpenCV (cv2), NumPy, scikit-image (used to source the sample image)

The sample image used throughout this lab (coffee) was obtained from the scikit-image built-in dataset (`skimage.data.coffee()`), as required by the lab instructions.

3. Procedural Steps

Step 1: Reading an Image

The image was loaded using `cv2.imread()`. Inspecting `img.shape` returned `(400, 600, 3)`, where:

- 400 = image height (rows)
- 600 = image width (columns)
- 3 = number of color channels, stored by OpenCV in **BGR** order (not RGB)



1.input

Step 2: Loading an Image in Grayscale

Using `cv2.imread(path, cv2.IMREAD_GRAYSCALE)`, the image was loaded directly as grayscale. The resulting shape was `(400, 600)` — only 2 dimensions, since grayscale images store a single intensity value per pixel instead of 3 color channels.

Step 3: Saving an Image

The grayscale image was saved to disk using `cv2.imwrite('images/output.jpg', gray_img)`.



2.output

Step 4: Converting Color Spaces

The color image was converted to grayscale using `cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)`. The image was also converted to the **YUV** color space using `cv2.cvtColor(img, cv2.COLOR_BGR2YUV)`, and the three channels were separated:

- **Y** – luminance (brightness)
- **U** – blue projection
- **V** – red projection

Each separated channel had shape (400, 600), since each represents a single intensity value per pixel.

4. Assessment – Task 1: Geometric Transformations

4.1 Resize (Increase Image Size)

The image was resized using a scale factor ($fx=1.5$, $fy=1.5$) rather than fixed dimensions, to preserve the original aspect ratio and avoid distortion.

- Original shape: (400, 600, 3)
- Resized shape: (600, 900, 3)



3. resized_output

4.2 Rotation (120 degrees)

The image was rotated by 120° around its center using `cv2.getRotationMatrix2D(center, 120, 1)` combined with `cv2.warpAffine()`. Black regions appeared in the corners of the output image, since after rotation those areas no longer correspond to any pixel data from the original image — this is expected behavior.



4. rotated_output

4.3 Shear Transformation

A horizontal shear was applied using a custom transformation matrix:

$$M = \begin{bmatrix} 1 & 0.2 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

The output frame width was increased to accommodate the slanted shape. The result was a parallelogram-shaped image with black background areas, consistent with the geometric nature of the shear operation.



5.sheared_output

5. Assessment – Task 2: Intensity Transformations

5.1 Negative Image

Computed as: $\text{new_pixel} = 255 - \text{old_pixel}$

Applied directly to the full image array using NumPy broadcasting: `negative_img = 255 - img`. This inverts all pixel intensities (dark areas become light and vice versa).



6. negative_output

5.2 Log Transformation

Formula: $s = c * \log(1 + r)$, where $c = 255 / \log(1 + 255)$.

The image was first converted to float64 to preserve precision during the logarithmic operation (since uint8 only supports integers and would cause overflow/rounding errors), then scaled back to uint8 after the transformation. The +1 inside the logarithm avoids the undefined value of $\log(0)$, since pixel values of 0 can genuinely occur in the image.



7. log_output

5.3 Power Law (Gamma) Transformation

Formula: $s = c * (r ^ \text{gamma})$, with $\text{gamma} = 2$ chosen to increase contrast.

Pixel values were normalized to the range [0, 1] before applying the power operation (since raising raw pixel values like 200 to the power of 2 would produce 40,000 — far outside the valid 0–255 range) and rescaled back to [0, 255] afterward.



8. power_output

6. Conclusion

This lab provided hands-on experience with fundamental image processing operations using OpenCV. Key concepts learned include the internal BGR pixel ordering used by OpenCV, the difference between geometric transformations (which alter spatial structure) and intensity transformations (which alter pixel values), and the importance of data type management (uint8 vs float64) and value normalization when performing mathematical operations on image arrays.